Experiment 4

Student Name: Jayanaath S
UID: 23BCC70022
Branch: AIT-CSE
Section: 23AIT_KRG-1 A
Semester: 6
Date of Performance: 06/02/2026
Subject Name: Full Stack II
Subject Code: 23CSH-382

- **Aim:**
  The aim of this experiment is to optimize the performance and user experience of the "Eco-Track" application by implementing memoization techniques, route-based code splitting, and a professional design system using Material UI components.

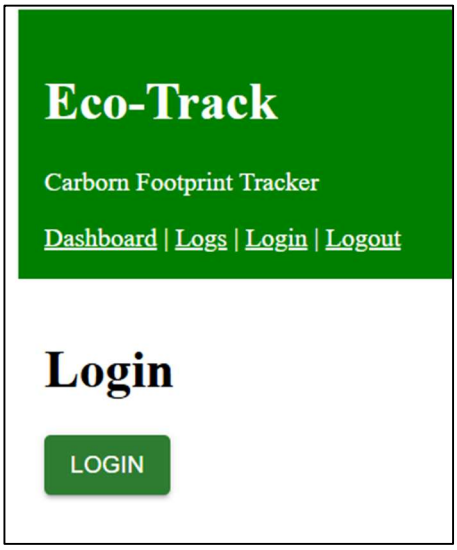
- **Objectives:**
  The main objectives of this experiment are as follows:
  1. Understanding Re-renders: Identifying and mitigating the causes of unnecessary component re-renders to ensure a fluid user experience.
  2. Component Memoization: Utilizing React.memo to skip re-rendering components when their props have not changed.
  3. Optimizing Calculations: Applying useMemo to cache expensive filtering and data transformation logic, specifically for high-carbon activity logs.
  4. Stable Function References: Implementing useCallback to memoize event handlers and prevent the breaking of child component optimizations.
  5. Code Splitting: Implementing dynamic imports with React.lazy and Suspense to load route-level components only when needed.
  6. Performance Feedback: Enhancing perceived performance by using Suspense boundaries with Material UI loading indicators during navigation.
  7. Enterprise UI Design: Integrating Material UI (MUI) components like CircularProgress, Button, and Box to create a professional, responsive interface.

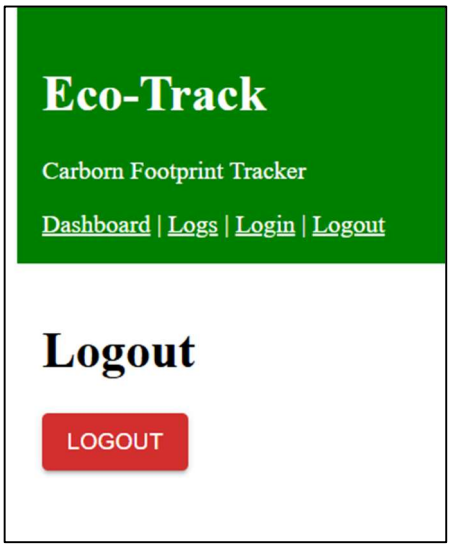
- **Implementation:**

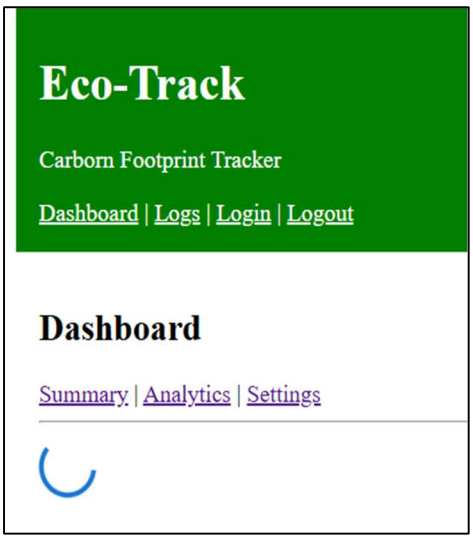
The development of the fourth iteration of Eco-Track followed these technical phases:

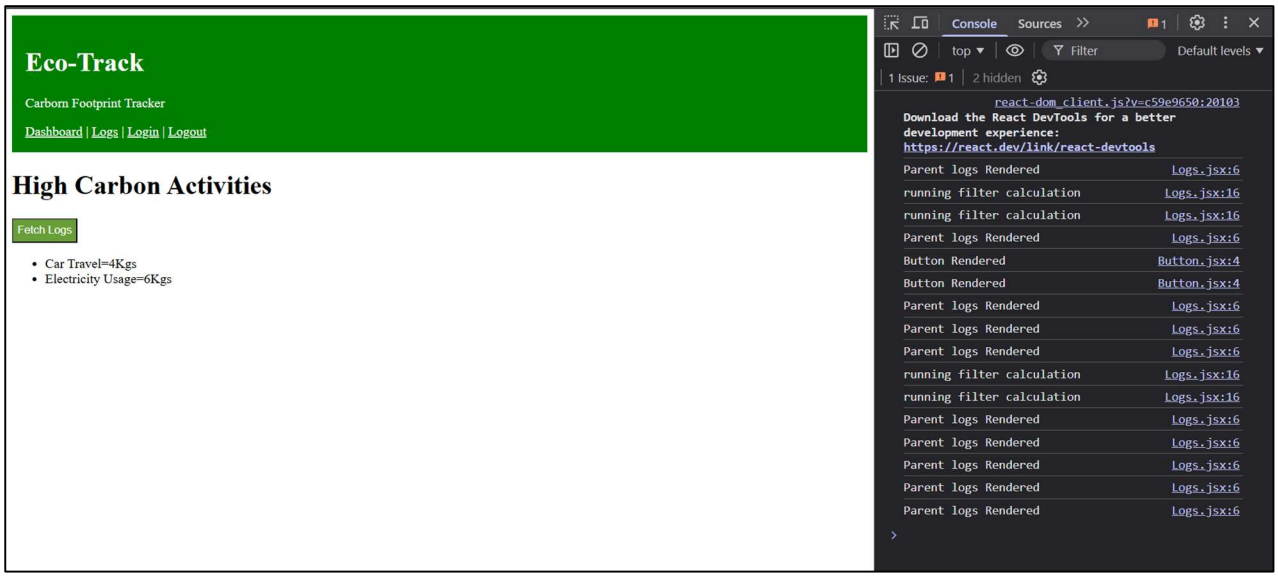
1. MUI Integration: The Material UI core library and Emotion styling engine were installed and integrated into the project.
2. Route Lazy Loading: Major page components (DashboardSummary, Analytics, Logs) were refactored using React.lazy() to reduce the initial bundle size.
3. Layout-Level Suspense: A <Suspense> boundary was implemented within the DashboardLayout, utilizing a location.pathname key to trigger immediate visual feedback during route transitions.
4. Loading States: Material UI CircularProgress component were used to create aesthetically pleasing loading fallbacks for async chunks.
5. Data Memoization: The Logs.jsx component was optimized by wrapping high-carbon filtering logic in useMemo, ensuring calculations only re-run when the Redux store data actually changes.
6. Function Memoization: The Redux dispatch trigger for fetching logs was wrapped in useCallback to maintain a stable function reference across re-renders.
7. Refined Component Structure: The custom Button component was refactored or replaced with MUI Button components, utilizing props like variant="contained" and color="success" to match the Eco-Track theme.

- **Output:**


Eco-Track
Carborn Footprint Tracker
Dashboard | Logs | Login | Logout
Login
LOGIN


Eco-Track
Carborn Footprint Tracker
Dashboard | Logs | Login | Logout
Logout
LOGOUT


Eco-Track
Carborn Footprint Tracker
Dashboard | Logs | Login | Logout
Dashboard
Summary | Analytics | Settings


Eco-Track
Carborn Footprint Tracker
Dashboard | Logs | Login | Logout
High Carbon Activities
Fetch Logs
Car Travel=4Kgs
Electricity Usage=6Kgs
Console
Sources
top
Filter
Default levels
1 Issue: 1
2 hidden
react-dom_client.js?v=c59e9650:20103
Download the React DevTools for a better development experience:
https://react.dev/link/react-devtools
Parent logs Rendered Logs.jsx:6
running filter calculation Logs.jsx:16
running filter calculation Logs.jsx:16
Parent logs Rendered Logs.jsx:6
Button Rendered Button.jsx:4
Button Rendered Button.jsx:4
Parent logs Rendered Logs.jsx:6
Parent logs Rendered Logs.jsx:6
Parent logs Rendered Logs.jsx:6
running filter calculation Logs.jsx:16
running filter calculation Logs.jsx:16
Parent logs Rendered Logs.jsx:6
Parent logs Rendered Logs.jsx:6
Parent logs Rendered Logs.jsx:6
Parent logs Rendered Logs.jsx:6
Parent logs Rendered Logs.jsx:6

- **Results:**
  The Eco-Track application was successfully transformed into a high-performance, enterprise-ready web app. By implementing code splitting, the initial load time was significantly reduced. Memoization techniques successfully eliminated redundant logic execution, resulting in a more responsive UI even as the data volume increases. The integration of Material UI shifted the design from a basic prototype to a professional-grade dashboard, demonstrating a deep understanding of React performance patterns and modern design systems.

- **Learning Outcomes:**
  After completing this experiment, I have learnt to:
  1. Identify Bottlenecks: Detect and fix unnecessary re-renders using React DevTools.
  2. Optimize Computation: Use useMemo to prevent expensive operations from running on every render.
  3. Stabilize Callbacks: Use useCallback to maintain referential equality for functions passed as props.
  4. Master Code Splitting: Use React.lazy and Suspense to improve application "Time to Interactive" (TTI).
  5. Leverage Design Systems: Build a consistent UI using Material UI layouts, typography, and progress indicators.
  6. Manage Route Lifecycle: Use useLocation and keys to control how React handles navigation states.